CITS3007 Secure Coding
Analysis and testing

Unit coordinator: Arran Stewart

Highlights

- (Automatic) program analysis
- Static analysis techniques
- Dynamic analysis techniques
- Fuzzing

Program analysis

What is it?

- the process of using automated tools to analyse the behaviour of computer programs for particular properties
  - e.g. correctness, security, speed, termination

Why analyse programs at all?

- Programs are complex and humans miss things
- Bugs are expensive to find late
- Security flaws often hide in edge cases
- Testing alone is not enough ...

Why isn't testing enough?

- Tests only cover *some* inputs
- Edge cases are easy to miss
- Some bugs depend on rare conditions
- Undefined behaviour may not show up reliably

Static vs Dynamic Analysis

What kinds of analysis exist?

- **Static analysis**: examine code without running it
- **Dynamic analysis**: observe behaviour at runtime

Static vs Dynamic analysis detail

- Static analysis: performed without executing the program.
  - Uses the program's *static* artifacts (usually, source code, but sometimes binary executables)
- Dynamic analysis: performed at runtime.
  - Actually runs the program (or part of it)
  - Typically *instruments* the code (adds extra instructions to the compiled code)
- Hybrid: a mix of the previous two.

How do they differ?

Static analysis

- Finds issues early
- Covers many paths (in principle)
- May report false positives

Dynamic analysis

- Observes real executions
- Precise for that execution
- Misses untested paths

Related techniques

If a *human* analyses the code, we don't usually call that "program analysis".

- In the static case, we just call it "reading the code", or "code review"
- In the dynamic case, we call it "debugging" or "manually running tests"

Related techniques

Automated versus non-automated isn't a sharp line – the borderline can be fuzzy.

e.g. Program verification isn't completely automated – it typically requires human input.

program verification

- Program verification asks: given a specification, can we *prove* that this program will always behave according to it, for every possible input and execution path?
- Tools used include model checkers, deductive verification languages or tools (Dafny, Frama-C), and interactive theorem provers (Rocq, Lean)

Analysis for C – pain points

Think of a bug you hit in C that took a long time to diagnose or fix.

- What made it difficult?
- When did the program actually go wrong?
- When did you notice?

Analysis for C – what would have helped?

- Detect the bug at the moment it occurs
- Identify the *type* of memory error
- Point to the exact line responsible

Dynamic analysis in practice

Dynamic analysis

Analyses which require the program to be *run* in order to work.

In C, the typical approach is to use a compiler plugin which injects extra instructions into the code when it is compiled

- These extra instructions help answer questions like "When we run the program with these particular inputs – do any of the array accesses go out of bounds?"

What tools exist for C?

Some of the most useful tools are:

- AddressSanitizer (ASan)
- UndefinedBehaviorSanitizer (UBSan)
- Valgrind (Memcheck)

(Plus various other Google Sanitizers)

Valgrind

Valgrind:

- Actually a suite of analyses – the most common is `memcheck` – plus a framework for developing new ones.
- Simulates the effect of every instruction in the original program (fairly slow)
- Doesn't require you to re-compile your code with instrumentation: uses the original binary, and inserts extra instructions (e.g. for out of bounds writes)

Dynamic analysis examples

Sanitizers, developed by Google

- code is actually part of LLVM, a compiler infrastructure framework
- the sanitizers:
  - AddressSanitizer (ASan) – detects buffer overflows and other, similar memory errors
  - ThreadSanitizer (TSan) – detects data races and deadlocks)
  - UndefinedBehaviorSanitizer (UBSan) – detects various sorts of undefined behaviour
  - plus many others
- incorporated into both the `clang` and `gcc` compilers
- require you to recompile your program and link to special libraries, so your code can be properly *instrumented* (i.e., have additional code inserted)

What do sanitizers give you?

- Detection of many types of memory bugs: use-after-free, buffer overflow, etc.
- Precise runtime error reports
- **Stack traces** to the root cause

```
=================================================================
==91892==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffeec7c4954 at pc 0x00010343be5
READ of size 4 at 0x7ffeec7c4954 thread T0
    #0 0x10343be4f in main overflow.cpp:5
    #1 0x7fff71ce27fc in start (libdyld.dylib:x86_64+0x1a7fc)

Address 0x7ffeec7c4954 is located in stack of thread T0 at offset 52 in frame
    #0 0x10343bd2f in main overflow.cpp:3

  This frame has 1 object(s):
    [32, 52) 'arr' (line 4) <== Memory access at offset 52 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapconte
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow overflow.cpp:5 in main
Shadow bytes around the buggy address:
  0x1fffdd8f88d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x1fffdd8f88e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x1fffdd8f88f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x1fffdd8f8900: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x1fffdd8f8910: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x1fffdd8f8920: 00 00 00 00 f1 f1 f1 f1 00 00[04]f3 f3 f3 f3 f3
  0x1fffdd8f8930: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Tradeoffs

Dynamic analysis can be very fast and precise.

- e.g. Google sanitizers will give very precise reports on where (say) undefined behaviour or out-of-bounds memory access occurs, and usually only add about 10-15% to runtime
- However, you're limited to only the code that was actually executed – you may have very limited *coverage*.
  - If you don't run your program with the right inputs, you may never discover a particular vulnerability

How to increase coverage?

Try to create thorough tests:

- Cover all entry points
  - Call every function your program exposes
- Include edge cases
  - Have you considered empty arrays, NULL pointers, zero-length strings?
  - Have you tried passing in the max/min values for integers?
- Vary input order and size
- Use automated scripts
  - Helps run the same tests repeatedly
  - Run tests with `-fsanitize=address,undefined` (Clang/GCC)
  - In labs, we will look at using libcheck[1] https://github.com/libcheck/check

[1] Actually, the name of the library is just "check". But that is very hard to search for, so almost everyone calls it "libcheck"

How to increase coverage?

- Target all branches
  - `if`, `switch`, loops, error conditions
- Keep tests reproducible
  - Use fixed seeds when randomness is involved
- Use different optimization levels
  - Compilers can actually emit **very different code** at different optimization levels – try all `-O0`, `-O`, `-O2`

How to increase coverage?

Another approach:

- instrumented fuzzing (more on this later)

Static analysis in practice

Dynamic versus static analysis

Recall:

- Static analysis: analysis performed without executing the program.

Static analysis can have perfect coverage (since the whole source code is available)

Doesn't need to run the program – may even be able to detect problems "as-you-type" (ALE in `vim` will do this)

However, answering some questions (e.g. "Will array accesses ever be made out of bounds?") are intractable for static analyses.

Would a static analyser catch this?

```
int *p = malloc(sizeof(int));
if (rand() % 2) {
  free(p);
}
*p = 42;
```

- Is there a bug?
- Could a static analyser reliably detect it?
- Why or why not?

Would a static analyser catch this?

```
int *p = malloc(sizeof(int));
if (rand() % 2) {
  free(p);
}
*p = 42;
```

- Depends on runtime behaviour (`rand()`)
- Requires reasoning about multiple execution paths
- Some properties are fundamentally undecidable

What does this mean in practice?

Static analysers must:

- Approximate behaviour
- Choose between:
  - Missing bugs (false negatives)
  - Reporting possible bugs that aren't real (false positives)

Static analysis examples

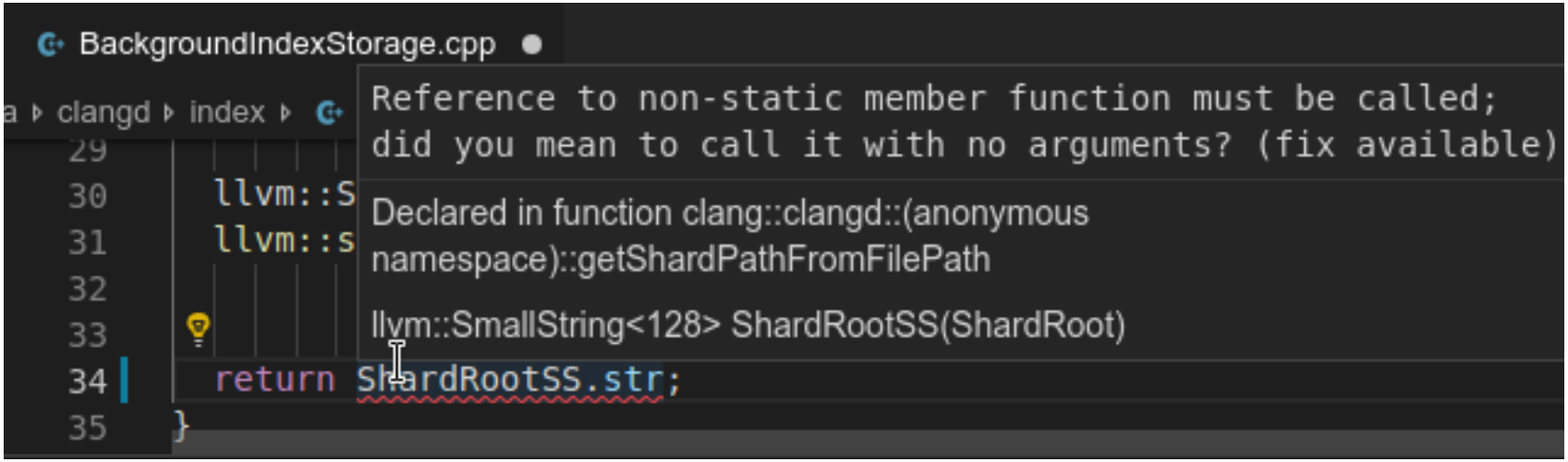
Some examples:

- Flawfinder
  - Developed by David A. Wheeler, director of security at the Linux Foundation
- clang-tidy
  - Created by the developers of the Clang compiler

We use both of these in the lab on "static analysis" – they warn about problematic code constructs.

Static analysers are also sometimes called "linters" or "bugfinders" (depending on their focus).

Analysis in the IDE



Many IDEs and editors provide ways of integrating warnings from static analysers into your development environment

- e.g. They may show warnings as red underlines in your code, with "hoverable" details

What tools and techniques exist?

Static analysis does cover a very wide range of techniques.

From very very simple – e.g. `grep`ping code for functions like `strcpy`, known to be unsafe – to very complex.

- Compiler warnings (`-Wall`, `-Wextra`)
- Linters (e.g. `cppcheck`)
- Clang Static Analyzer

Targets of static analysis

- Many static analysis programs operate on the source code for a program, but some instead analyse compiled binaries.

For example, the Ghidra framework can be used to analyse binary code.

We won't use any binary analysis techniques in this unit, but they can come in handy when doing penetration testing and reverse engineering.

What do static analysers do well?

- Detect obvious bugs early
- Enforce coding rules
- Catch issues before running code

What do they struggle with?

- Complex control flow
- Pointer aliasing
- Input-dependent behaviour
- Reducing false positives

What are the limits of static analysis?

Static analysis cannot be as precise as dynamic.

```
if halts(f):
  reveal_all_the_secrets()
```

Static analyses *approximate* the behaviour of the program: they provide either false positives or false negatives.

False positive Reporting a program has some property (e.g. a vulnerability) when it does not

False negative Reporting a program does not have some property when it does

Soundness and completeness

Capabilities of static analyses are often described in terms of **soundness** and **completeness**.[2]

An analyser can be sound in relation to some runtime property of a program.

> **sound** If the analyser asserts that a program has property P, then the program *does* have property P. The analyser is never wrong about this.

[2] The terms comes from logic. "sound" ≈ "says true things", "complete" ≈ "all true things are said")

Soundness

sound If the analyser asserts that a program has property P, then the program *does* have property P. The analyser is never wrong about this.

But:

- It might be "over-scrupulous" – sometimes say that programs which *do* have property P, actually do not.
- An analyser could be trivially sound, by simply *never* saying a program has property P – it would never be wrong...

Soundness example

- Suppose a static analyser, *DivDetector*, analyses programs and tries to assess whether they have property *DivGood*: "At runtime, this program will never encounter a division-by-zero exception".
- When fed a program, DivDetector may say "Yes, it has property DivGood" or "No, it does not".
  (Some analysers might also say "I don't know", or remain silent.)

✓ If DivDetector never lies when it says "Yes, this program has property DivGood", then it is *sound*.

✘ But it might say "No, this program does *not* have property DivGood", and be wrong.

Soundness example

If we frame the problem as "identify programs with property DivGood", then DivDetector can suffer from false negatives – it sometime might say a program *doesn't* have property DivGood, when it does.

Completeness

complete If a program has property P, then the analyser always detects it.
(An analyser could be trivially complete, by saying *everything*. e.g. It says "This program is correct", *AND* "This program is incorrect".)

But:

- It may be "over-eager" – might sometimes say that programs which *don't* have property P, actually do.
- It could be trivially complete, by saying *every* program has property P – it would never "miss" a program.

Compromises

What we would *like* is for an analysis to be both sound and complete – it says exactly all the true things.

But it is impossible for any algorithm to determine (for all programs) any non-trivial semantic property of a program – this is Rice's theorem.

So instead, analysers will have to compromise on one or the other (or both)

- Sometimes they say a program has some property P, but it doesn't
- Sometimes a program will *have* property P, but the analyser won't report it

Compromises

They might also require certain assumptions about the code.

e.g. A static analyser might be sound, when detecting some property of Java programs, *as long the program doesn't use reflection*. (If it does, the analyser might give wrong answers, and "let in" some programs which don't actually have the property.)

(This is actually quite common. For instance, there are programs for analysing Python code, *as long as functions like* `eval()` *aren't used*.)

Example – compilers

For statically typed languages, the compiler guarantees that type errors won't occur at runtime.

But it does so by forbidding many programs in which type errors never would have occurred at runtime.

```
// java method
void do_a_thing() {
  int n = 1;
  if (1 + 1 == 3) {
    n == "hello";
  }
}
```

Example – compilers

```
// java method
void do_a_thing() {
  int n = 1;
  if (1 + 1 == 3) {
    n == "hello";
  }
}
```

Compiler writers would probably say the question being asked is, "Is the program well-typed (i.e., no type errors occur at runtime)?"

The compiler is *sound* (if it says a program is well-typed, it is), but incomplete (sometimes a program would actually be well-typed, but the compiler says it is not).

Example – compilers

But it's just as valid to flip the question, and ask "Will the program exhibit a type error at runtime?".

Then the compiler is *complete* – when type errors occur, they are always reported – but it is not *sound* (sometimes the compiler says a type error would occur, when it actually won't).

Reporting (e.g. security) errors

Does a program have property *GoodProperty* (e.g. *BadThing* never happens)?

	complete	incomplete
sound	Reports all occurrences (If *GoodProperty* is there, analyser says it exists) No false alarms (If analyser says *GoodProperty* exists, it does exist) **Undecidable**	May miss occurrences (If *GoodProperty* is there, analyser may or may not say so) No false alarms (If analyser says *GoodProperty* exists, it does exist) **Decidable**
unsound	Reports all occurrences (If *GoodProperty* is there, analyser says it exists) Possible false alarms (Analyser reports property, but it may not exist) **Decidable**	May miss occurrences (If *GoodProperty* is there, analyser may or may not say so) Possible false alarms (Analyser reports property, but it may not exist) **Decidable**

Decidability

"Undecidable" means no *algorithm* (guaranteed to terminate) exists.

But if we wanted, instead of compromising on soundness or completeness, we could compromise on terminability. (Sometimes, the algorithm might run forever).

(Example: some type checker algorithms *are*, in fact, not guaranteed to terminate; though generally, it turns out that for all "normal" programs written by and of interest to humans, they do end up terminating.)

Static analysis in practice

Since perfect solutions aren't possible

- analysers give *approximate* results (compromise on soundness or completeness), or
- require manual assistance, or
- have timeouts (analysis could in theory run forever)

Static analysis in practice

In general, programmers dislike false positives:

- analyser reports many "problems", most of which are false alarms

We will see an example of this in the lab with Flawfinder and clang-tidy.

Many of the signed-to-unsigned conversions reported are probably harmless, but all the reports obscure bigger problems.

Sorts of static analysis

Static analysers for many different tasks:

- Type checking
- Style checking
- Property checking (ensuring some property holds – e.g. no deadlocks, bad behaviour of some sort)
- Program verification (ensuring correct behaviour w.r.t some specification)
- Bug finding (detecting likely errors)

Sorts of static analysis

- Type checking
- Style checking
- Property checking (ensuring some property holds – e.g. no deadlocks, race conditions, bad behaviour of some sort)
- Program verification (ensuring correct behaviour w.r.t some specification)
- Bug finding (detecting likely errors)

All are useful.

- Type checking prevents runtime errors
- Style checking makes code easier for humans to review
- Property checking can avoid some security bugs
- Program verification (doesn't ensure our *design* or use of e.g. crypto functions is correct though – just correctness w.r.t spec)
- Bug finding (use for secure coding is obvious)

Type systems

Powerful type systems can provide very strong guarantees about program behaviour.

- Memory-safe languages (Java, ML, Haskell, Rust): memory corruption (if the language is properly implemented, and programmer restricted to safe parts of language) impossible

But

- All these languages do, in practice, provide "escape hatches" (if nothing else, they all allow C routines to be called which aren't themselves type-checked)
- Programmers may regard type system as annoying and overly restrictive

Type systems

Statically checked type systems are *modular*

- small pieces can be checked
- the pieces are put together, and the interfaces are checked

One "holy grail" of type system + security research: are there type systems which would provide compositional guarantees for security properties?

Style checking

Type systems are *part* of the language, but style checking covers *good practice*.

Usually covers

- coding standards (layout, bracketing)
- naming conventions (e.g. `snake_case`, `camelCase`, `SCREAMING_SNAKE_CASE`)
- checking for dubious code constructs (e.g. in Python, use of `eval()`)

Example tools:

- `clang-format`, `clang-tidy` (C and C++)
- `pylint`, `black` (Python)
- `checkstyle`, `findbugs`, `PMD` (Java)
- `ShellCheck` (Bash)

Style checking

Most style checkers allow you to customize what rules are applied, and to exclude particular files, functions or lines from being checked

e.g. for pylint:

```
pylint: disable=some-message
```

Language-based security

Language-based security

The idea here is to use language features to check for application-level attacks

One example: taint tracking.

Can be static or dynamic

We add security labels to data inputs (sources) and data outputs (sinks).

tainted Data from unsafe sources (e.g. user input)

Or data derived from or influenced by tainted data

untainted Data we can safely output or use

Taint tracking

To switch something from tainted to untainted, it has to go through particular *sanitization* functions.

Examples:

- Perl provides a "taint mode" (https://perldoc.perl.org/perlsec)
- Taintdroid – modified Android runtime, tracks data flows at runtime (https://github.com/TaintDroid)

Type-checking information flow

Idea: define a type system which tracks security levels of variables in the program, and adding levels to sources and sinks. Security levels may be:

High

- Sensitive information, e.g., personal details
- Any other data that
  - is computed directly from high data
  - occurs in a high context (high test in if)

Low

- Public information, e.g., obtained from user input

Fuzzing

Dynamic analysis – fuzzing

- Many test cases are input into the target application
- Application is monitored for errors (e.g. crashes, hangs)

Types of fuzzing

- Mutation based ("dumb")
- Generation based ("smart")
- Evolutionary

Mutation Based Fuzzing

- Little or no knowledge of the structure of inputs
  - Not good for inputs with checksums, challenge responses, etc.
- New inputs may be completely random or follow some
- Very easy to set up
- Depends on inputs provided

Generation Based Fuzzing

- Test cases generated from some description, rules or grammar
- Knowledge of input structure usually gives better results than mutation
- Usually trickier to set up

Evolutionary Fuzzing

- Attempts to generate inputs based on the response of the program

Putting the techniques together

Static vs dynamic: how to use them?

- Static analysis: early, cheap, broad
- Dynamic analysis: precise, runtime, concrete
- They complement each other

For your C projects

- Always compile with warnings enabled
- Use sanitizers during development
- Fix warnings early
- Don't ignore suspicious reports

Key points

- No single technique is enough
- All analysis involves trade-offs
- Good developers use multiple tools together

Further reading

- OWASP discussion and list of static analysis tools
- Rival and Yi, *Introduction to Static Analysis* (2020, MIT Press)
- Zeller et al, *The Fuzzing Book* (available online)